



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application		
Course	Programming from First Principles		
Topic Name	Filter - Practical Session		
Topic Code	-		
Unit/Module No. & Name	5		Higher Order Functions
Self-Learning Hours	-		

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6 - 8
3.0	Nature Or Type	9 - 12
4.0	Examples And Case Studies	13 - 16
5.0	Keywords And Touch Vocabulary	17



OBJECTIVE

1. The objective of this practical session is to explain the concept of filter in simple language while keeping an academic style suitable for undergraduate students.
2. Filter is an important programming tool used when a collection contains many items, but only some of them are needed for the next step.
3. In such situations, filter helps select the items that satisfy a given condition and remove the rest.
4. This topic is important because students often work with lists of numbers, strings, records, marks, prices, and text values, and they need a clear method to keep only relevant data.
5. The session connects theory with practice. It explains what filter means, how it works, where it is used, why it is useful, and what care students should take while applying it in code.
6. It also links filter with first-class functions, lambda functions, loops, and data processing.
7. After studying this session, students should be able to define filter, explain its working pattern, apply it to simple programming problems, compare it with loops, and understand when filter is the correct choice in practical programming.
8. The session aims to build academic understanding, logical thinking, and coding confidence through this concept.

1.0 INTRODUCTION

In programming, students often work with collections that contain many items. These items may be numbers, names, marks, prices, dates, or records. In real tasks, however, the programmer does not always need every item. Sometimes only a small group of useful items is needed. For example, from a list of marks, the student may want only passing marks. From a list of prices, the user may want only prices above a certain value. From a list of names, the programmer may want only names that begin with a particular letter. In all such cases, the aim is not to change every item or combine all items into one answer. The aim is to keep some items and remove others.

This is where filter becomes useful. Filter is used when data must be selected on the basis of a rule. Each item is checked against a condition. If the item satisfies the condition, it is kept. If it does not satisfy the condition, it is removed from the result. In this way, filter helps the programmer work with only the required data.

Filter is best understood through practical examples. If students only memorize the definition, the idea may seem too abstract. But if they see a list such as 10, 25, 30, and 12 and apply a rule like keep values above 20, the result becomes easy to understand. The output clearly shows how some values remain and others disappear. This visual difference between input and output helps learning.

A practical session also helps students compare filter with other common programming patterns. A loop can also be used to inspect each item and keep only selected values. However, filter expresses this selection task more directly. It allows the student to focus on the condition rather than on the repeated loop structure. By comparing filter with loops, students improve both their coding skill and their programming judgment.

At the undergraduate level, students should understand more than syntax. They should understand the logic behind the tool and the type of problem it solves. Filter is academically important because it teaches the idea of condition-based selection. This idea appears in many parts of computer science, including data cleaning, database queries, search operations, reporting, data analysis, and validation.

Filter also helps students think in terms of problem patterns. Some tasks need transformation. Some tasks need reduction. Some tasks need selection. Filter teaches the selection pattern clearly. Once students understand this, they are better prepared to study map, reduce, comprehensions, query logic, and advanced data handling techniques in later subjects.



2.0 MEANING AND DEFINITION

2.1 Meaning Of Filter

The word filter means to separate useful or required items from a larger group. In everyday life, a filter removes unwanted material and allows needed material to pass through. In programming, the idea is similar. Filter checks each item in a collection and allows only matching items to remain in the result. Items that do not match the required condition are excluded.

This meaning is easy to remember when students imagine a gatekeeper. Each item reaches the gate. The condition acts as the rule at the gate. If the item passes the rule, it enters the result. If not, it stays out. This step-by-step checking is the central idea of filter.

2.2 Formal Definition

Filter is a higher-order operation that applies a Boolean condition function to each item of an iterable and returns only those items for which the condition is true. In simple classroom language, filter checks all items one by one and keeps only the items that satisfy the given rule.

This definition includes several important ideas. First, filter is a higher-order operation because it takes a function as input. Second, it works on an iterable such as a list, tuple, or other collection. Third, the input function returns true or false. Fourth, the output contains only selected items, not all items. Because of this, the size of the output may be smaller than the size of the input.

2.3 The Condition Function

The function given to filter is the condition rule. It checks one item and returns a Boolean result. If the result is true, the item is included in the final output. If the result is false, the item is removed. This condition function is the heart of the filter process.

2.3.1 One Item At A Time

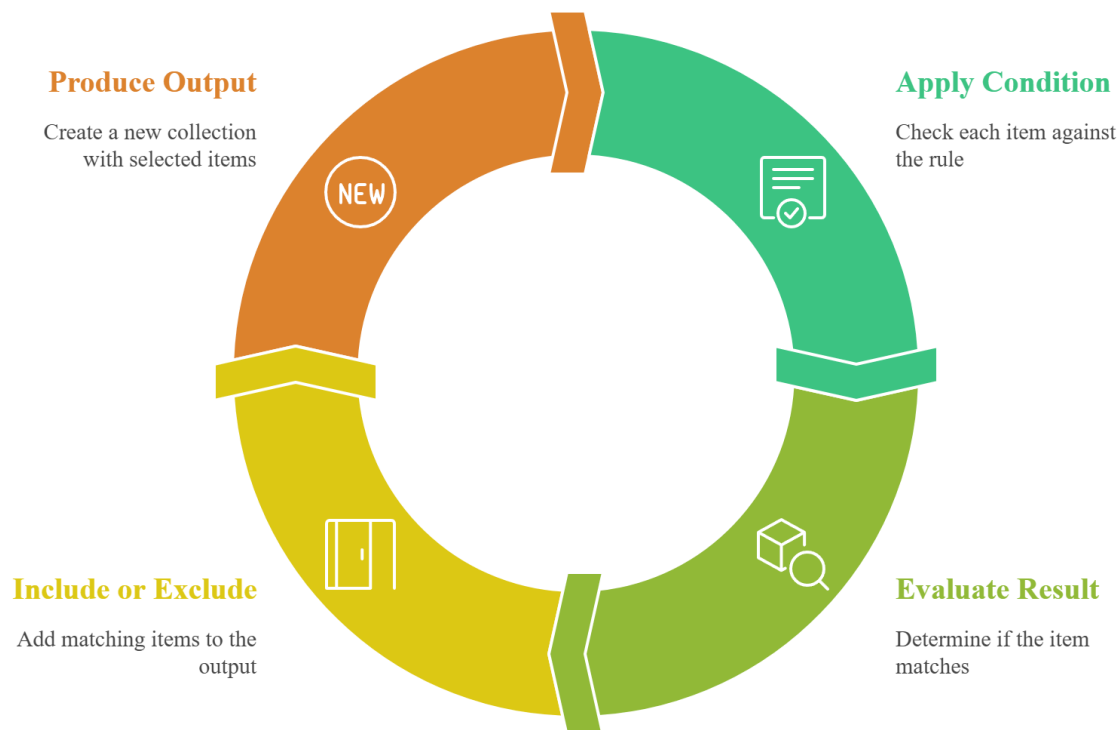
In basic filter usage, the condition function usually accepts one item at a time. It examines that item and decides whether it should remain in the output. This one-item approach keeps the process simple and clear.

2.3.2 True Or False Decision

The condition function must make a decision. It does not normally transform the item into a new value. Its main job is to answer a question such as: Is this value greater than 50? Does this name start with A? Is this number even? Is this mark a passing mark? The answer must be true or false.

Fig. 1

The Filtering Cycle



This circular diagram illustrates the filtering cycle, showing steps like applying conditions, evaluating results, including or excluding items, and producing a new filtered output collection.

2.4 Input Collection And Output Collection

Filter receives a collection of input items. It examines them in order and produces a new collection containing only the matching items. This means the output is usually smaller than the input, though sometimes all items may pass and sometimes no item may pass.

This feature helps students clearly distinguish filter from map and reduce. Map usually keeps the same number of items and changes each one. Reduce produces one final result. Filter keeps only selected items.

2.5 Relation With First-Class Functions

Filter is closely linked with first-class functions because the selection rule is passed as a function. If the rule is simple, a lambda function may be used. If the rule is long or reused many times, a named function is often better. This teaches students that functions can describe behavior for other tools, not only perform direct tasks on their own.

2.6 Filter Versus If Statement In A Loop

A loop with an if statement can also select matching items. So why study filter? The answer is that filter clearly expresses the pattern of selection. Instead of writing the full loop logic every time, the student can focus on the rule and let the filter process do the repeated checking. This supports abstraction. However, students should also understand that loops remain useful and are sometimes easier to read for complex conditions.



3.0 NATURE OR TYPE

3.1 Nature of the Filter

The nature of filter is selective, functional, reusable, and condition-based. Selective means it keeps only some items and removes others. Functional means it uses a function as the selection rule. Reusable means the same filter structure can solve many different tasks by changing the condition. Condition-based means every item is judged according to a rule.

This nature makes filters very useful in programming because many problems involve choice. A large collection may contain both useful and useless values. A filter helps separate them cleanly and systematically.

3.1.1 Selective Nature

Filter is selective because it does not keep everything. It examines items and includes only those that satisfy the given rule. This is the most important nature of the filter operation.

3.1.2 Functional Nature

Filter belongs strongly to the functional style of programming. A programmer provides a condition function, and the filter process applies it to each item. This shows how functions can be used as values and passed into other operations.

3.1.3 Uniform Nature

Filter is uniform because the same condition is applied to every item in the collection. This means the selection process is consistent. If the condition is keep even numbers, that rule is applied equally to all values in the input.

3.1.4 Non-Transformational Nature

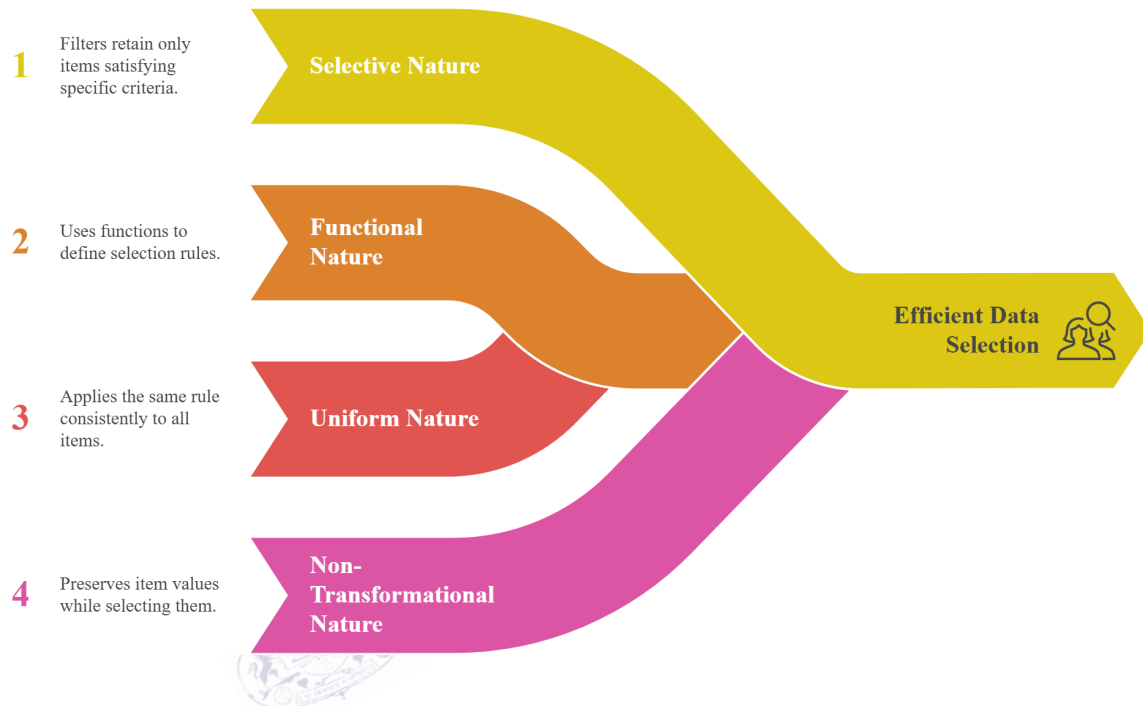
Filter does not mainly change the values themselves. Its main purpose is to decide which values should stay and which should leave. This helps students understand that filter is different from map, which transforms values.

3.2 Types Or Forms Seen In Practice

Filter is one general process, but in practical learning students see several common forms.

Fig. 2

The Essence of Filtering



This diagram explains the essence of filtering, highlighting its selective, functional, uniform, and non-transformational nature, emphasizing how it efficiently selects data based on defined conditions.

3.2.1 Numerical Filtering

This form selects numbers based on a condition. Examples include even numbers, odd numbers, values above a limit, values below a limit, or values inside a range. This is often the first type shown in class because it is easy to test.

3.2.2 Text Filtering

Text filtering selects strings based on rules such as length, first letter, presence of a word, or case pattern. This is useful in text processing, search tasks, and simple data cleaning.

3.2.3 Record Filtering

In record filtering, the input contains structured items such as tuples or dictionaries. The rule checks one field in each record. For example, it may keep only students with marks above 60 or only products with price below 1000. This type is useful in applications.

3.2.4 Logical Filtering

Logical filtering works with conditions that produce true or false results from complex tests. For example, a program may keep only values that are positive and even, or keep only names longer than five letters and starting with a capital letter. This shows that the condition can include more than one check.

3.2.5 Data Quality Filtering

This form is used in data cleaning. A filter rule may remove empty values, negative ages, missing records, or invalid entries. This is common in data preparation and analysis tasks.

3.3 Advantages Of Filter

Filter can make code shorter and more focused when the task is clear selection. It helps students think about the condition rather than the loop structure. It works well with lambda functions for short rules. It also improves readability when the programmer wants to show clearly that the main goal is to keep matching items.

Another advantage is that filter supports real data work. In practical problems, not all data is useful. Some data must be removed before the next step. Filter gives a systematic method for this selection.

3.4 Limitations And Care Points

Filter is not always the best answer. If the condition is very long or difficult to understand, a loop with named steps may be clearer. Students should also remember that filter only selects items. If the task is to change items, map may be a better choice. If the task is to combine items, reduce may be more suitable.

Another care point is type matching. The condition must suit the data. A numeric rule should not be applied to text unless the logic is designed properly. Students should therefore understand both the input and the condition before using filter.

3.5 Academic Rule For Proper Use

A good academic rule is this: use filter when the task is to keep only those items that match a clear condition and when the condition can be written in a readable way. Use a loop when the condition becomes too complex or when step-by-step explanation is more important than compact form.

4.0 EXAMPLES AND CASE STUDIES

4.1 Example: Keeping Even Numbers

The easiest practical example is selecting even numbers from a list. Suppose the input list contains 1, 2, 3, 4, 5, and 6. The condition checks whether each number is divisible by 2. The output becomes 2, 4, and 6. This example is useful because the rule is simple and the result is easy to verify by hand.

The example also helps students see that filter does not change the values. It only decides which values remain.

4.2 Example: Selecting Marks Above A Passing Limit

Suppose a list contains marks such as 25, 40, 67, 39, and 88. If the passing limit is 40, the condition keeps only values that are 40 or above. The output becomes 40, 67, and 88. This example connects the filter concept with academic data that students already understand.

It also shows the idea of threshold-based selection. Many real tasks involve threshold values, such as pass marks, salary limits, or price limits.

4.3 Example: Filtering Names By First Letter

Filter can also work with text. Suppose the input contains names such as Amit, Neha, Anjali, and Rohan. A condition may keep only names that begin with the letter A. The output becomes Amit and Anjali. This example helps students understand that filter is useful in text processing and search tasks.

4.4 Example: Filtering Long Words

A list of words may be filtered to keep only those with more than five letters. If the input contains data, science, code, and filter, the output becomes science and filter. This example shows how text can be selected by length.

4.5 Example: Using Lambda With Filter

In many practical cases, the condition is short. A lambda function is then a convenient way to define the rule. For example, a lambda function may check whether a number is positive, whether a mark is above 50, or whether a word starts with a given letter. This shows how lambda and filter work together.

Students should also remember that lambda is best for simple conditions. If the rule becomes large, a named function should be preferred for readability.

4.6 Example: Filtering Positive Numbers

If a list contains negative, zero, and positive values, filter can keep only the positive values. This example is useful in mathematics, science, and data cleaning tasks. It also shows that filter can remove unwanted values that may cause confusion in later processing.

4.7 Example: Filtering Non-Empty Text

In data entry work, some text items may be empty or contain only spaces. A filter condition can remove such values and keep only meaningful text. This example shows how filter supports data quality improvement before other operations are applied.

Case Study 1: Student Performance Report

Consider a student performance system that stores marks for many students. A teacher may want a list of only passing students, only high-performing students, or only students below a warning line. Filter can help create these selected groups quickly. The rule may check marks, attendance, or both.

This case study is important because it shows how filter supports academic reporting, analysis, and decision making. Students can see a direct use of the concept in educational systems.

Case Study 2: Product Selection In E-Commerce

An online store may have a large list of products. A user may want only products below a certain price, only products in stock, or only products from a particular category. Filter allows the

program to produce this smaller and more useful list. This case study shows that filter is important in user-facing systems and business applications.

Case Study 3: Data Cleaning In Surveys

Survey data often contains invalid or incomplete entries. Some ages may be negative. Some names may be empty. Some responses may be missing. Filter can remove such invalid rows before analysis begins. This makes later results more accurate and meaningful.

This case study helps students see that filter is not only for simple exercises. It is a practical tool for preparing real data.

Case Study 4: Attendance Monitoring

Suppose a college system stores attendance percentages for many learners. The department may want a list of students with attendance below 75 percent for warning notices. A filter condition can check each percentage and keep only those that fall below the limit. This case study shows how filter supports administration and academic monitoring.

Case Study 5: Text Search Preparation

A text processing system may store many words, phrases, or documents. The program may need to keep only lines containing a certain keyword, or only words longer than a limit. Filter can do this selection before the next analysis step. This is useful in search, indexing, and basic natural language tasks.

- Filter selects only those items that satisfy a condition.
- It uses a Boolean rule that returns true or false.
- The output size may be smaller than the input size.
- Filter is a higher-order operation because it accepts a function.
- The function can be written as a named function or a lambda function.
- Filter is different from map because it does not mainly change values.
- Filter is different from reduce because it does not combine all items into one result.
- It is useful for numbers, text, records, and data cleaning tasks.

- Filter works best when the condition is clear and readable.
- A loop may be better when the condition becomes too complex.
- Good use of filter improves abstraction and code clarity.
- Students should test small examples and predict the output manually.
- Filter is widely used in real systems such as search, reporting, and validation.



5.0 KEYWORDS AND VOCABULARY

KEYWORD	SIMPLE MEANING
Filter	A method that keeps only matching items
Condition	The rule used to decide the selection
Boolean	A true or false value
Iterable	A collection processed item by item
Higher-Order Function	A function that uses another function
Lambda Function	A short anonymous function
Named Function	A regular function with a formal name
Input Item	The current value being checked
Output Item	A value that remains after selection
Selection	Choosing needed items from a group
Threshold	A limit used for keeping or removing values
Data Cleaning	Removing invalid or unwanted data
Record Filtering	Selecting structured data items
Text Filtering	Selecting text values by a rule
Numerical Filtering	Selecting numbers by a rule
Loop	A repeated control structure
Abstraction	Focusing on the main idea and hiding repetition
Validation	Checking whether data follows rules
Search Logic	Finding required items from larger data
Readability	How easy code is to understand